

Как выбрать инструмент для трейсов в .NET?

Николай Савенко

.NET Разработчик, Altenar

Дата



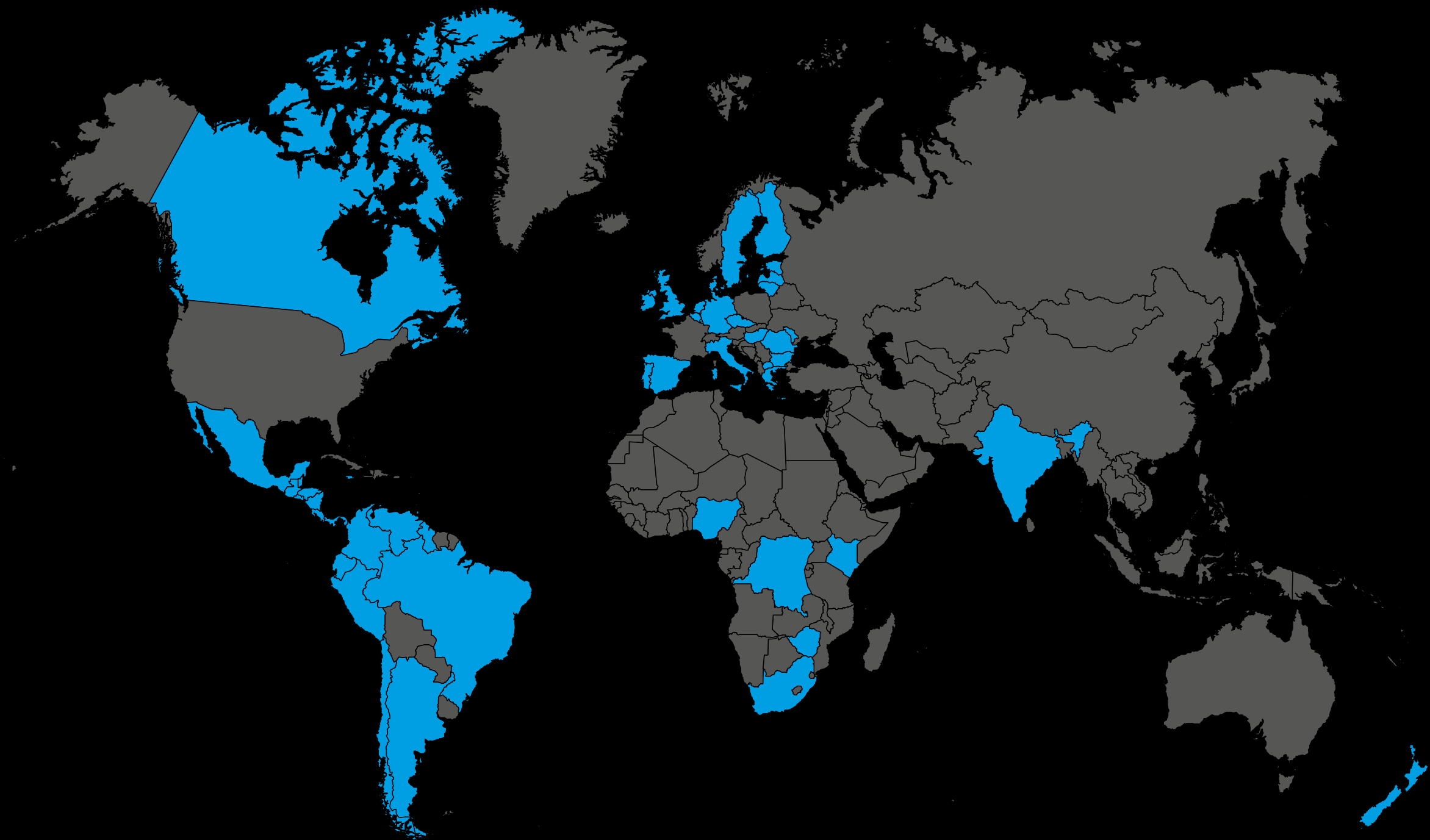
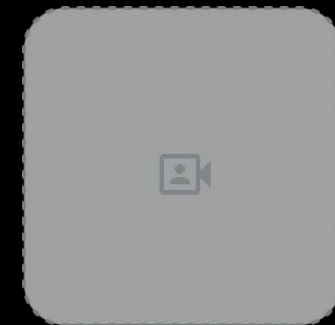
Николай Савенко

.NET Developer



Can break production at scale
Developer 12/10





10+ лет

на зарубежном рынке B2B ПО
для легальных операторов
прогнозов на спорт

780+ сотрудников

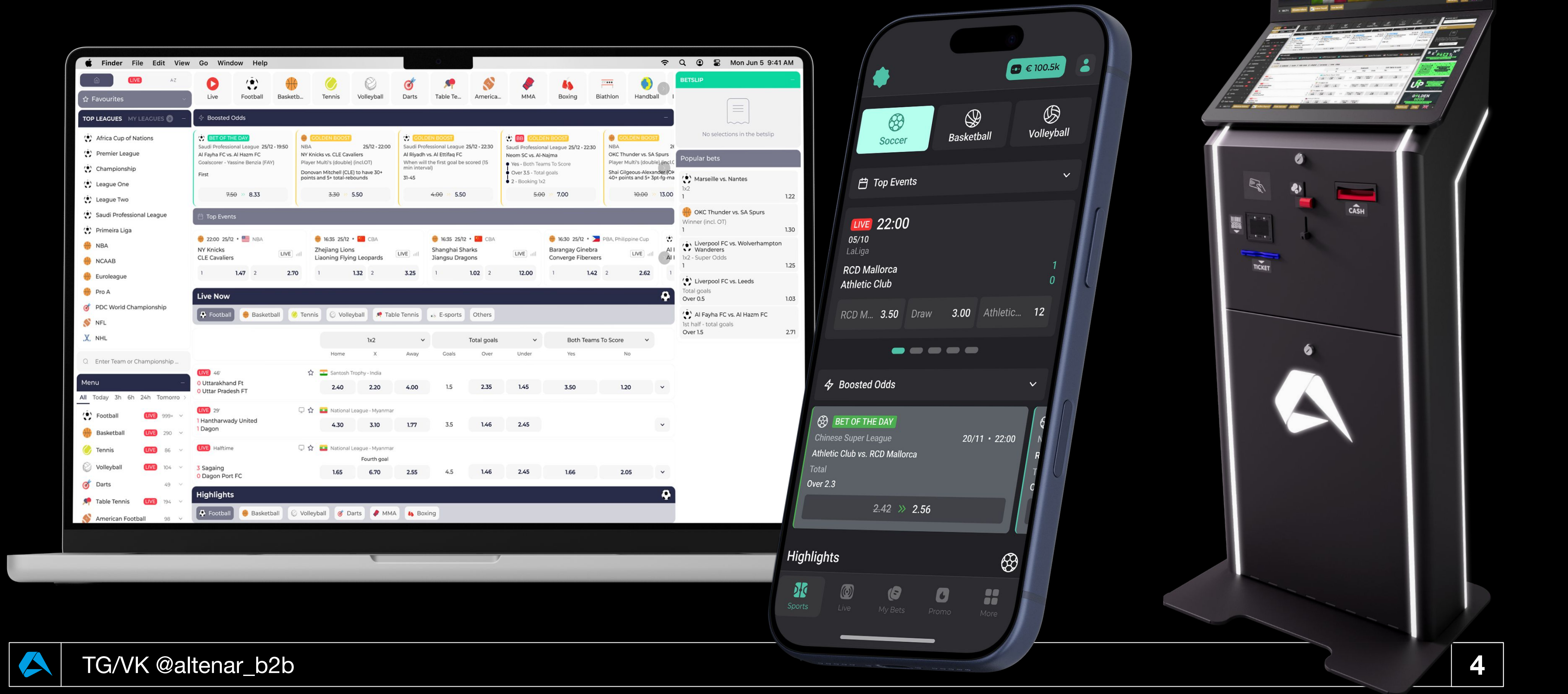
на удаленке и в офисах
в 6 странах

60k+ RPS

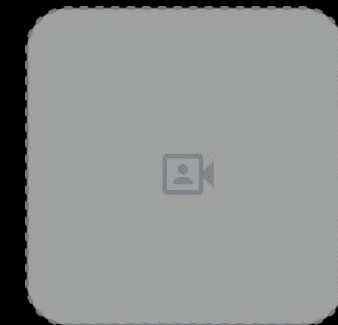
обрабатывают наши API
в дни Лиги чемпионов



Порталы, мобайл, ПО для терминалов



TG/VK @altenar_b2b



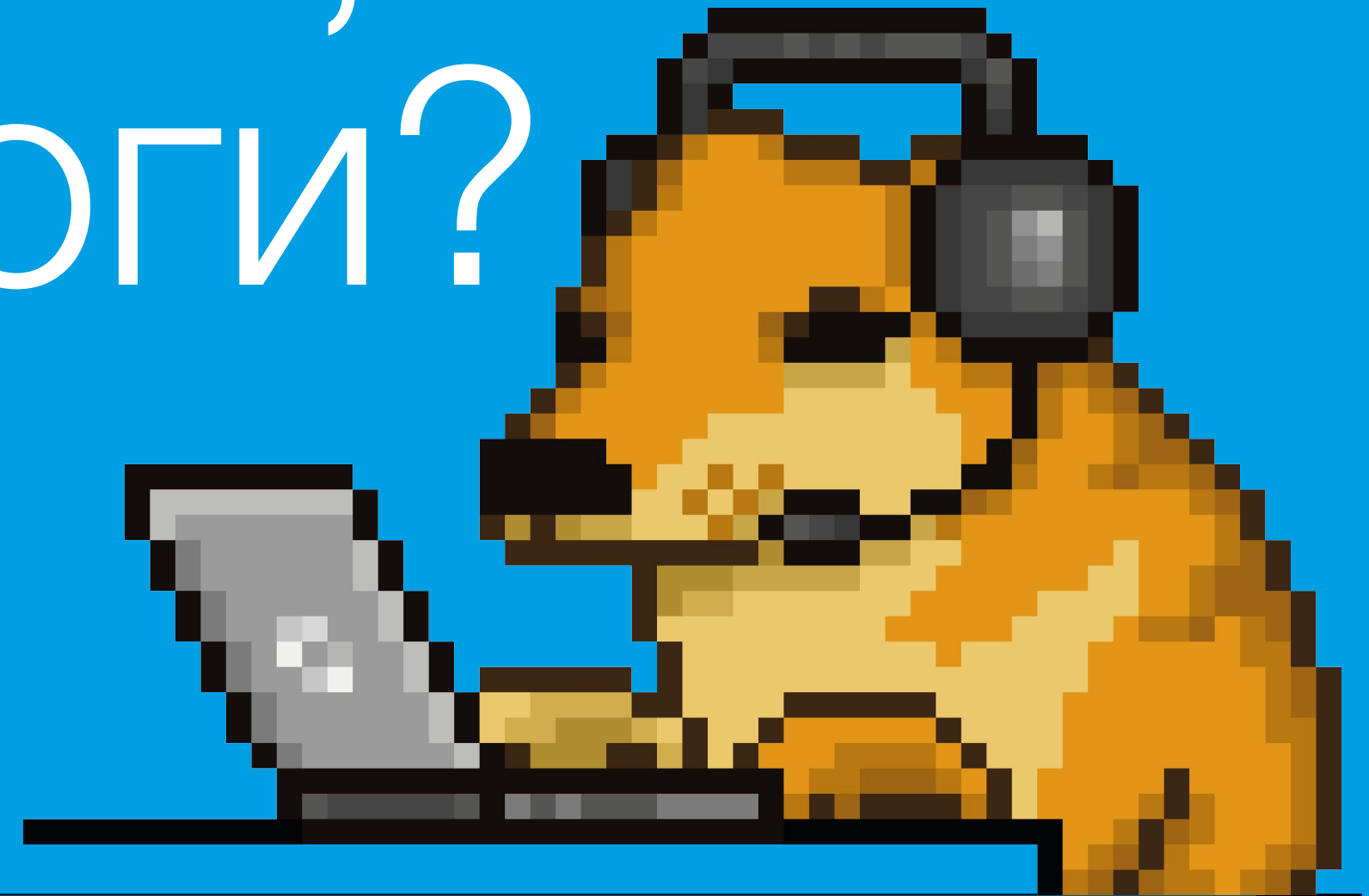
Что мы сегодня узнаем?



- Как мониторили раньше?
- Чем отличаются?
 - *ActivitySource* и как его готовить
 - *DiagnosticSource*
 - *EventSource*
- Как внедрить в свой код?
- Как внедрить в чужой код?
- Какие могут быть проблемы?



Зачем трейсы, если есть логи?



Задача логов и трейсов

Логи:

- События в одном сервисе
- Одиночные операции или состояния
- Временная метка и уровень
- Нет информации о зависимостях между сервисами.

Пример:

“Transfer 200 funds from user A to user B”.

Трейсы:

- Связанные шаги в разных частях приложения.
- Parent-child связи между span'ами.
- Визуализируют всю цепочку выполнения.

Пример:

“HTTP запрос → обработка в сервисе → вызов БД → отправка сообщения в очередь”.



Чего мы хотим от трейсинга?

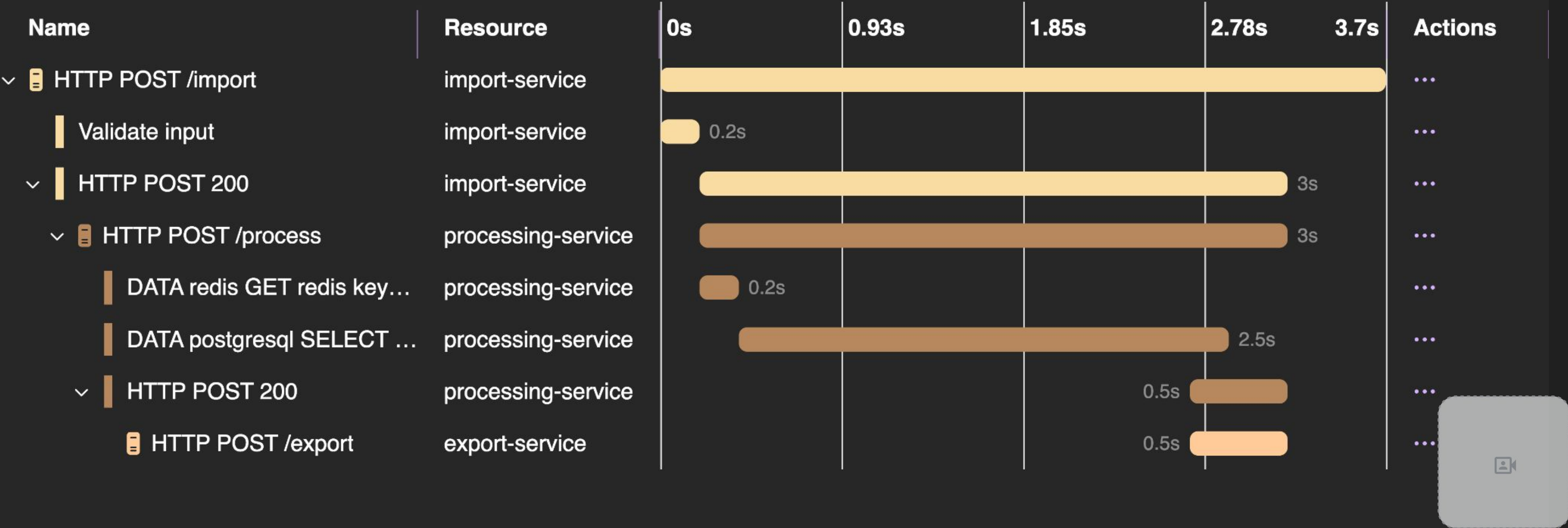


- Что за операция?
- Длительность?
- Связь с другими операциями?
- Распределенность
- Визуализация

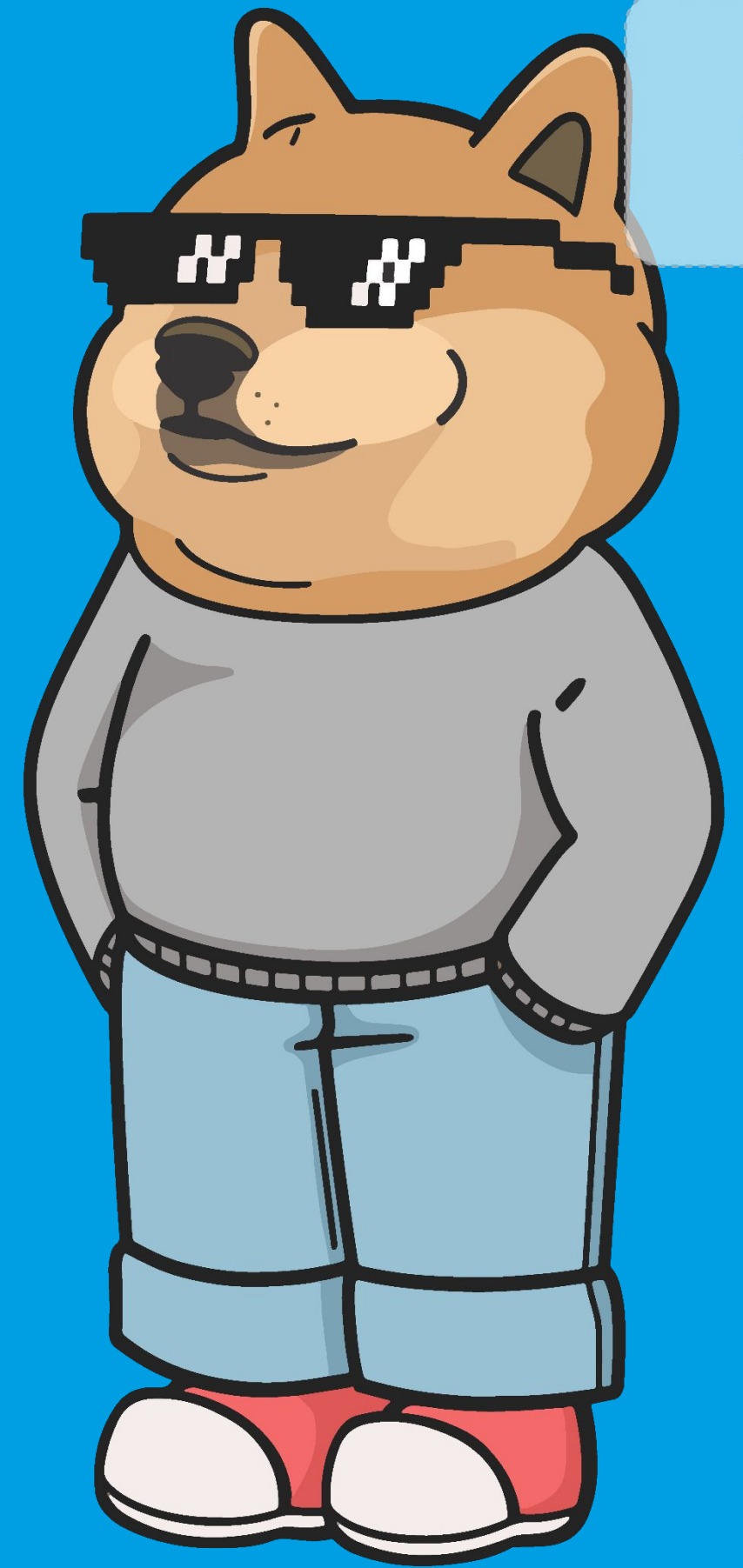
Ожидания: 😎

import-service: HTTP POST /import eba71a7

Trace detail 3/9/2024 7:00:00.000 PM Duration 3.7s Resources 3 +2 Filter... Type (All) ⚙️



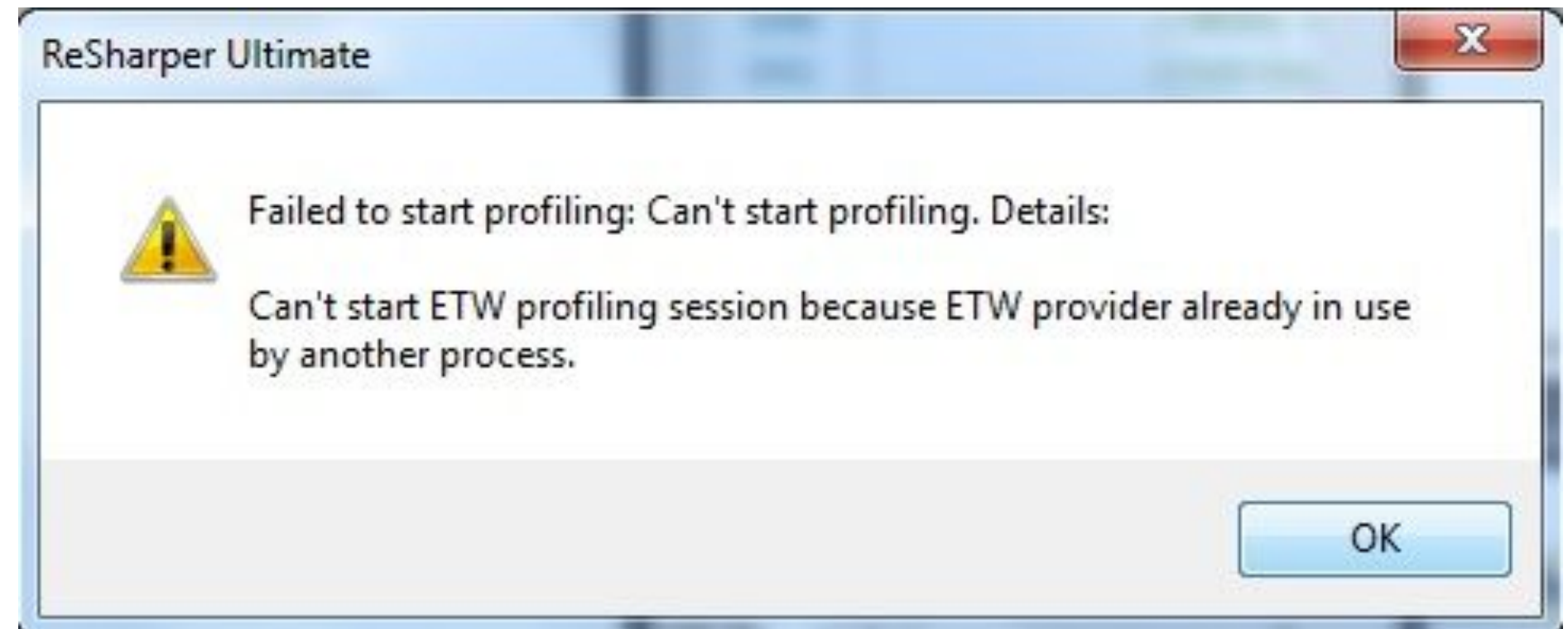
ЧТО МЫ ИМЕЕМ В .NET?



EventSource - когда трава была зеленой...



- ETW = Event Tracing for Windows
- Single Host
- Low Level API



EventSource - только события



```
[EventSource(Name = "MyCompany-MyLibrary")]
public sealed class MyEventSource : EventSource
{
    public static readonly MyEventSource Log = new();

    [Event(1, Message = "Processing with foo={0}, bar={1}", Level = EventLevel.Informational)]
    public void SomeWork(string foo, int bar) ⇒ WriteEvent(1, foo, bar);
}

static async Task DoSomeWork(string foo, int bar)
{
    MyEventSource.Log.SomeWork(foo, bar);

    await StepOne();
    await StepTwo();
}
```



Создаем свой EventListener



```
class ConsoleWriterEventListener : EventListener
{
    protected override void OnEventSourceCreated(EventSource eventSource)
    {
        if (eventSource.Name == "MyCompany-MyLibrary")
        {
            EnableEvents(eventSource, EventLevel.Informational);
        }
    }

    protected override void OnEventWritten(EventWrittenEventArgs eventData)
    {
        Console.WriteLine(eventData.Timestamp + " " + eventData.EventName);
    }
}
```

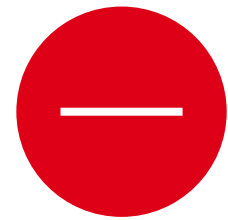


Создаем экземпляр Listener



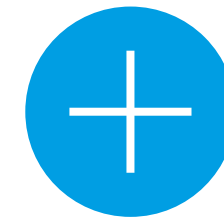
```
public static void Main(string[] args)
{
    ConsoleWriterEventListener listener = new ConsoleWriterEventListener();

    DemoEventSource.Log.SomeWork("Hello!", "World!");
}
```



Недостатки

- Не предназначен для distributed tracing
- Много шаблонного кода
- Не для бизнес событий



Преимущества

- низкий overhead
- устоявшийся API

Порешали вопросы?



- Что за операция? 
- Длительность? 
- Связь с другими операциями? 
- Распределенность 
- Визуализация 

DiagnosticSource - шаг вперед



```
static readonly DiagnosticListener listener = new("MyCompany.MyLibrary");

static async Task DoSomeWork(string foo, int bar)
{
    if (listener.IsEnabled("SomeWork"))
    {
        var payload = new { foo, bar };
        listener.Write("SomeWork.Start", payload);

        await StepOne();
        await StepTwo();

        listener.Write("SomeWork.Stop", payload);
    }
    else
    {
        await StepOne();
        await StepTwo();
    }
}
```



Как подписаться на нужный?

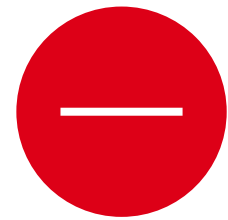


```
DiagnosticListener.AllListeners.Subscribe(listener =>
{
    if (listener.Name == "MyCompany.MyLibrary")
    {
        listener.Subscribe(new MyDiagnosticObserver());
    }
});
```



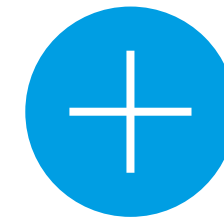
```
class MyDiagnosticObserver : IObserver<KeyValuePair<string, object>>
{
    public void OnCompleted() ⇒ throw new NotImplementedException();
    public void OnError(Exception error) ⇒ throw new NotImplementedException();

    public void OnNext(KeyValuePair<string, object> value)
    {
        var payload = (MyPayload)value.Value;
        Console.WriteLine($"Example {payload.GetType().Name} ");
    }
}
```



Недостатки

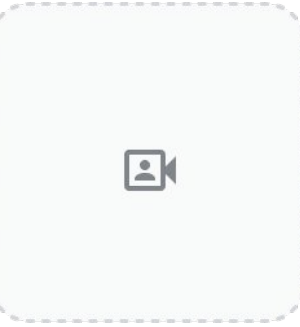
- Нет встроенной поддержки иерархической трассировки



Преимущества

- Можно передавать сложные структуры данных
- Есть начало и конец

Стало лучше?



- Что за операция? 
- Длительность? 
- Связь с другими операциями? 
- Распределенность 
- Визуализация 

ActivitySource



ActivitySource - актуальный стандарт



```
static readonly ActivitySource source = new("MyCompany.MyLibrary");
```

```
static async Task DoSomeWork(string foo, int bar)  
{
```

```
    using Activity activity = source.StartActivity("SomeWork");
```

```
    activity?.SetTag("foo", foo);
```

```
    activity?.SetTag("bar", bar);
```

```
    await StepOne();
```

```
    await StepTwo();
```

```
}
```



Как подписаться?



```
var listener = new ActivityListener
{
    ShouldListenTo = (source) => source.Name == "MyCompany.MyProduct",

    Sample = (ref ActivityCreationOptions<ActivityContext> options) => ActivitySamplingResult.AllData,

    ActivityStarted = activity => Console.WriteLine($"Started: {activity.OperationName}"),
    ActivityStopped = activity => Console.WriteLine($"Stopped: {activity.OperationName}")
};

// Register
ActivitySource.AddActivityListener(listener);
```

Почему Span называются Activity?



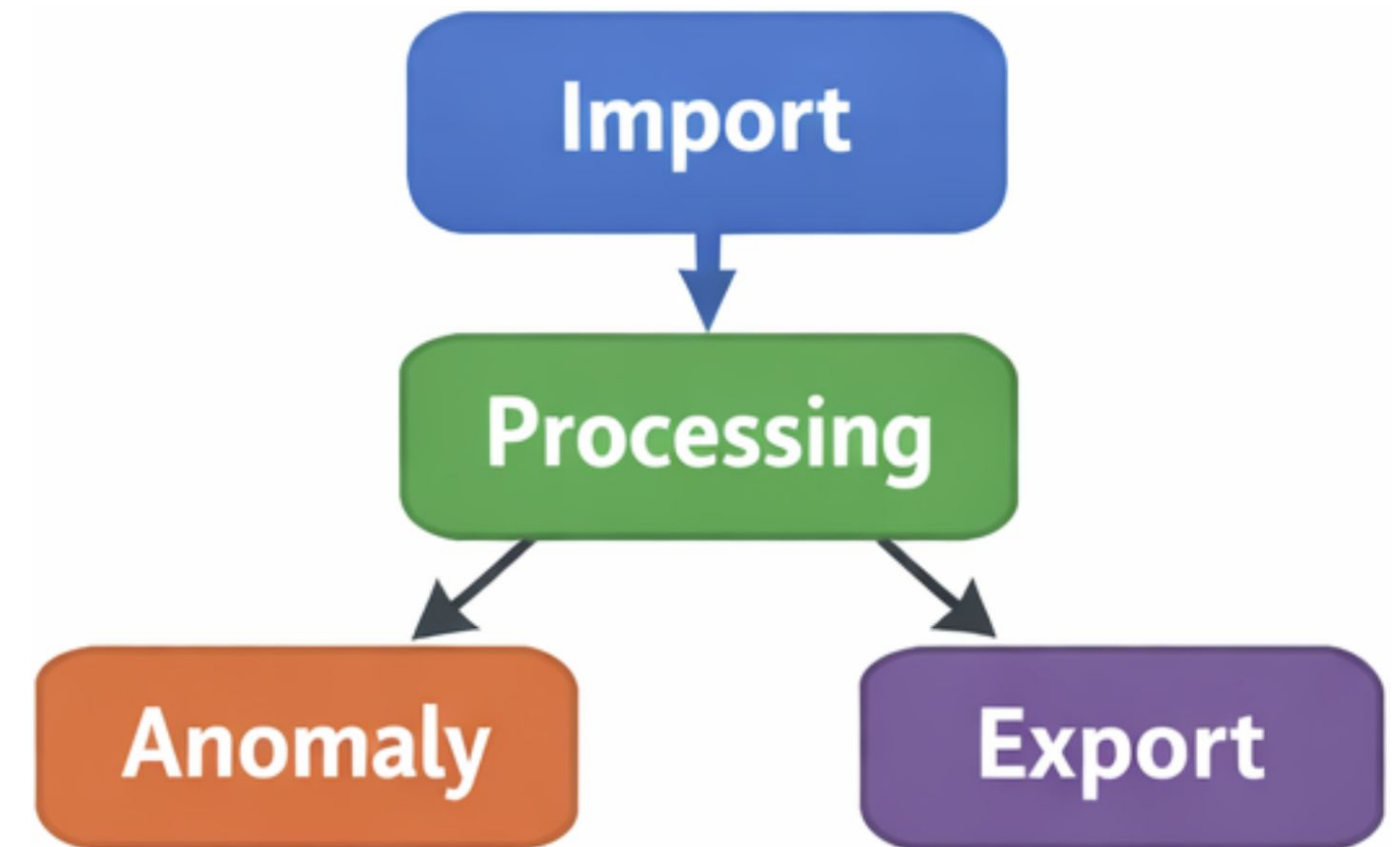
ⓘ Note

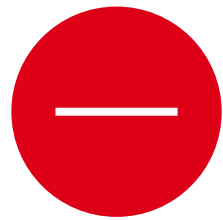
Another common industry name for units of work in a distributed trace are 'Spans'. .NET adopted the term 'Activity' many years ago, before the name 'Span' was well established for this concept.

Связанность разных Activity



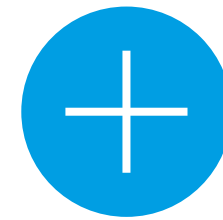
```
using var parent = source.StartActivity("Import");  
using var processing = source.StartActivity("Processing");  
  
using var anomaly = source.StartActivity(  
    "Anomaly",  
    parentContext: processing.Context);  
  
using var export = source.StartActivity(  
    "Export",  
    parentContext: processing.Context);
```





Недостатки

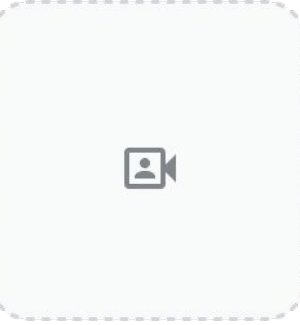
- Неявный контекст - `Activity.Current`
- Нет “Trace” как объекта
- Сложность тестирования



Преимущества

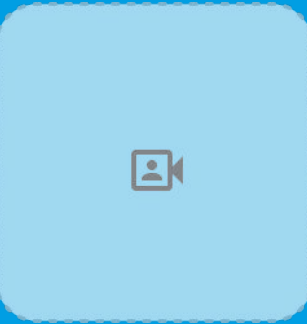
- Поддержка Distributed Tracing
- Простота внедрения
- Otel из коробки

А сейчас все закрыли?

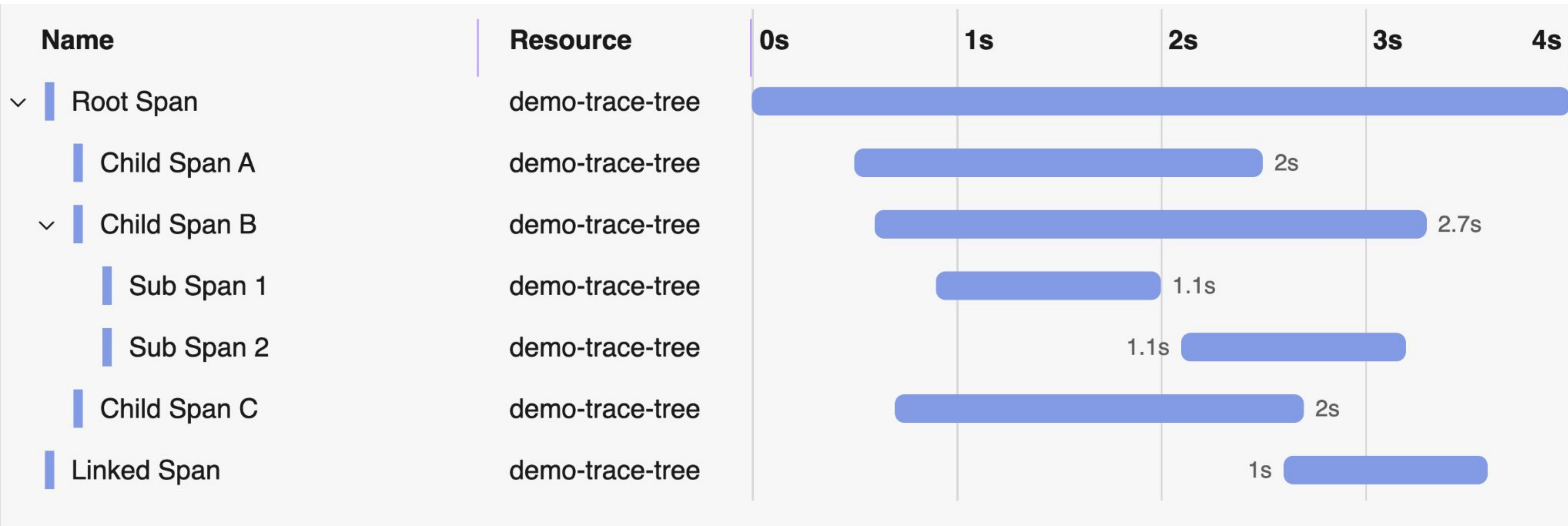


- Что за операция? 
- Длительность? 
- Связь с другими операциями? 
- Распределенность 
- Визуализация 

Distributed Tracing

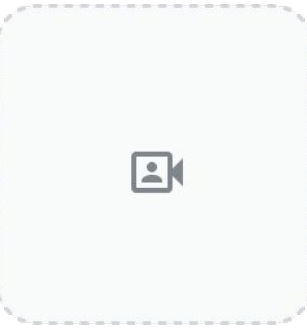


Все Activity связаны через TraceContext



Где находится контекст?

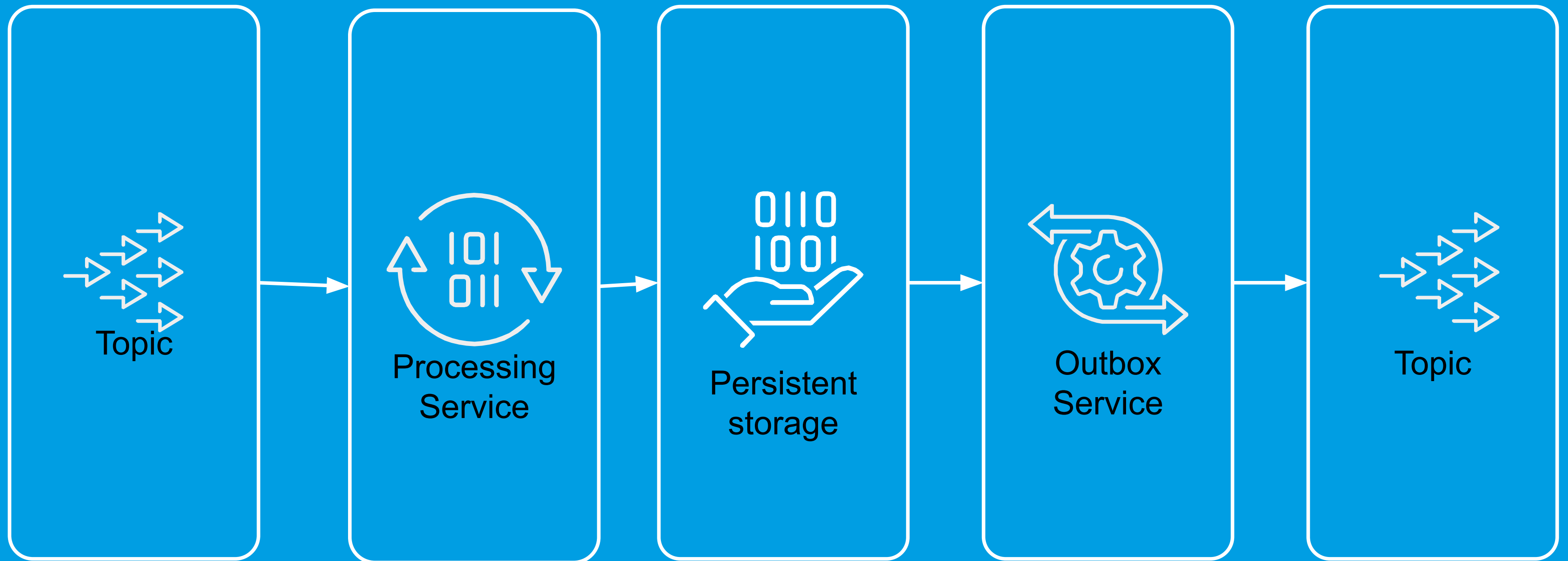
- HTTP Headers
- Message Headers
- Database columns
- etc...



Context Propagation



Transactional Outbox наносит ответный удар



Как достичь Context Propagation?



- TraceParent
 - *version-traceid-spanid-flags*
- Example
 - *00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01*

// Service A

```
var traceparent = Activity.Current.Id;
```

// Service B

```
if (ActivityContext.TryParse(traceparent, tracestate: null, out var parentContext))  
{  
    using var activity = MyActivitySource.StartActivity(  
        "SendMessage",  
        ActivityKind.Producer,  
        parentContext);  
    activity?.SetTag("messaging.system", "kafka");  
}
```



Почти победа



- Что за операция? 
- Длительность? 
- Связь с другими операциями? 
- Распределенность 
- Визуализация 

Визуализация?



На любой вкус и цвет



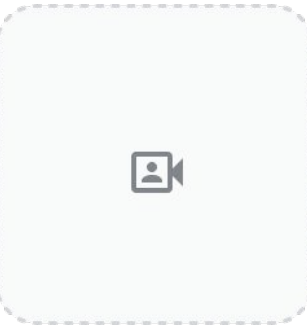
ZIPKIN



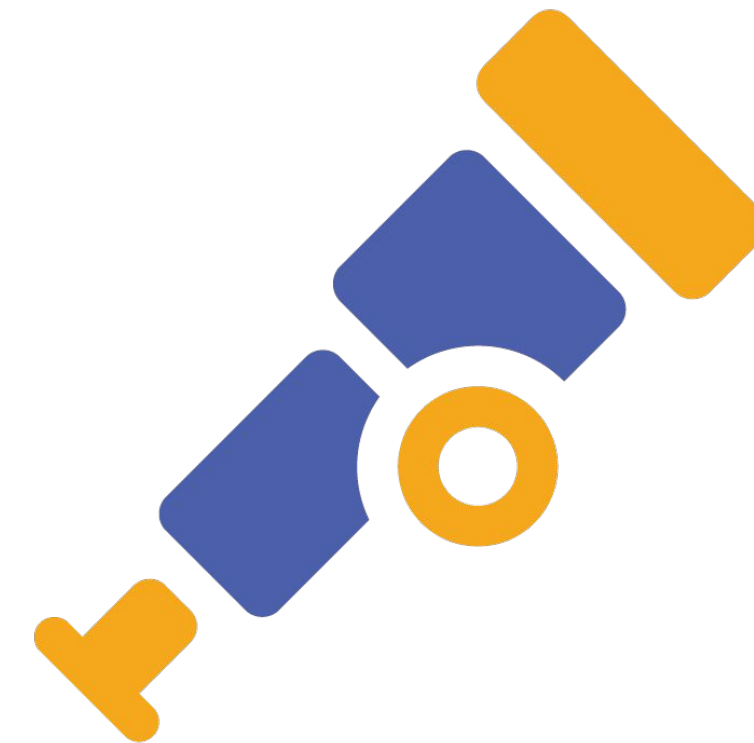
Vendor Lock



Один протокол, чтоб править всеми

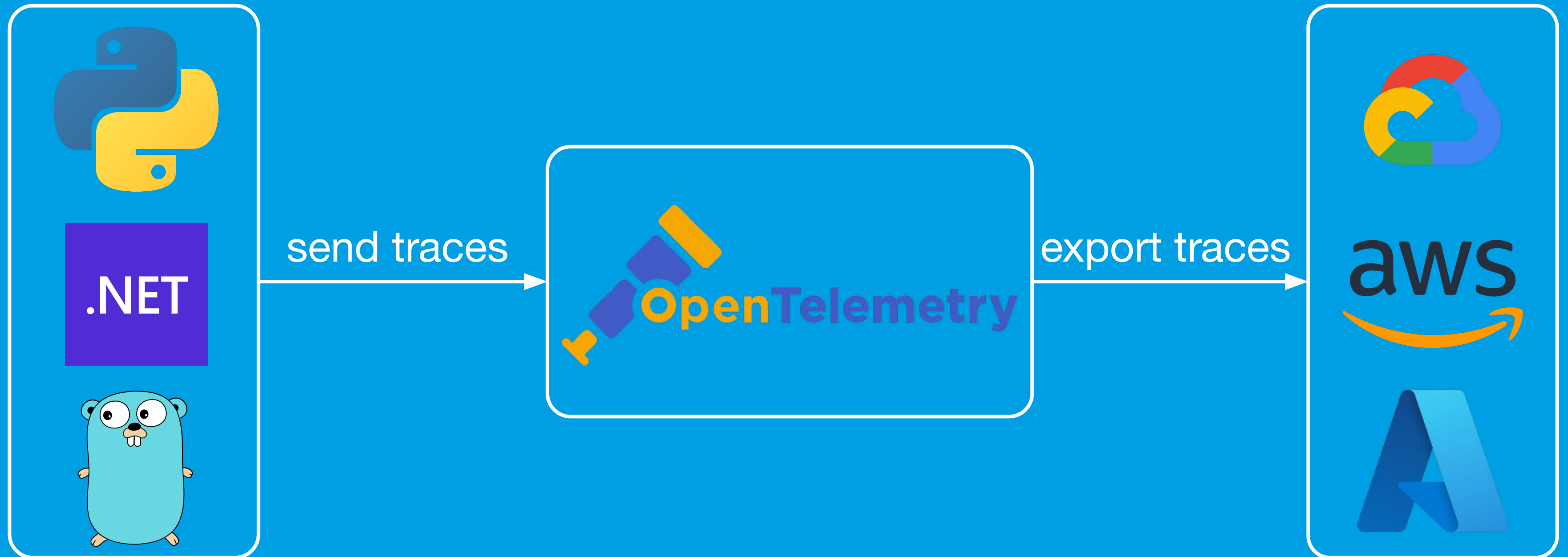
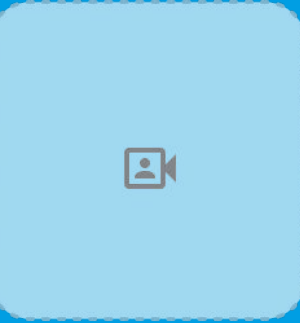


- Logs
- Metrics
- Traces



OpenTelemetry

ActivitySource + OpenTelemetry = ❤️



OpenTelemetry + Google Cloud Trace



Name Collapse all	Service/workload ?	0s86.285ms172.57ms258.854ms345.13
▼ Trace ID 0a49adced662b8bf09cf10accf0a36f...		345.139ms
▼ message_handle	LineBlocking	69.483ms
▼ SendMessage	Generic.Outbox.Service.Kafka	10ms
▼ ProduceAndRetryIfNeededAsync	Generic.Outbox.Service.Kafka	7.317ms
ConsumeAndRetryIfNeededAsync	MatchBlocker	19.6µs
▼ SendMessage	Generic.Outbox.Service.Kafka	10.328ms
▼ ProduceAndRetryIfNeededAsync	Generic.Outbox.Service.Kafka	7.663ms
ConsumeAndRetryIfNeededAsync	Export.Markets.Soccer	14.2µs

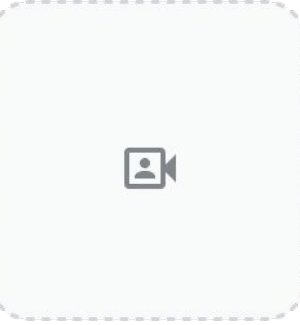


Ожидания: 😎

import-service: HTTP POST /import eba71a7



А дальше что?



- Что за операция? 
- Длительность? 
- Связь с другими операциями? 
- Распределенность 
- Визуализация 

Хотим трейдинг для gRPC, EF, HttpClient





Как обстоят дела в реальности?



Все уже перешли на ActivitySource?



До сих пор на DiagnosticSource:

- Entity Framework Core
- System.Net.Http.HttpClient
 - Активно создает Activity, но не через ActivitySource
- SqlClient
- MongoDB.Driver
- gRPC
 - до .NET 7 использует DiagnosticSource
- MassTransit, MediatR, Hangfire, Quartz.NET, etc...



✓ Сценарий 1: Компонент использует ActivitySource



```
builder.Services.AddOpenTelemetry()  
    .WithTracing(builder =>  
    {  
        builder  
            .AddSource("ComponentName")  
            .AddOtlpExporter(); // или другой экспортер  
    });
```



Сценарий 2: DiagnosticSource и есть библиотека instrumentation



```
builder.Services.AddOpenTelemetry()  
    .WithTracing(builder =>  
    {  
        builder  
            .AddXyzInstrumentation()  
            .AddOtlpExporter();  
    });
```

🤪 Сценарий 3: Нет готовой библиотеки инструментарии



```
DiagnosticListener.AllListeners.Subscribe(listener =>
{
    if (listener.Name == "Component.Name")
    {
        listener.Subscribe(new MyDiagnosticObserver());
    }
});
```

😬 Сценарий 3: Нет готовой библиотеки instrumentation



```
// MyDiagnosticObserver.cs
public void OnNext(KeyValuePair<string, object?> value)
{
    using var activity = MySource.StartActivity("SomeOperation",
        ActivityKind.Internal);
    activity?.SetTag("custom.tag", value.Key);
}
```


😬 Сценарий 4: Компонент использует EventSource



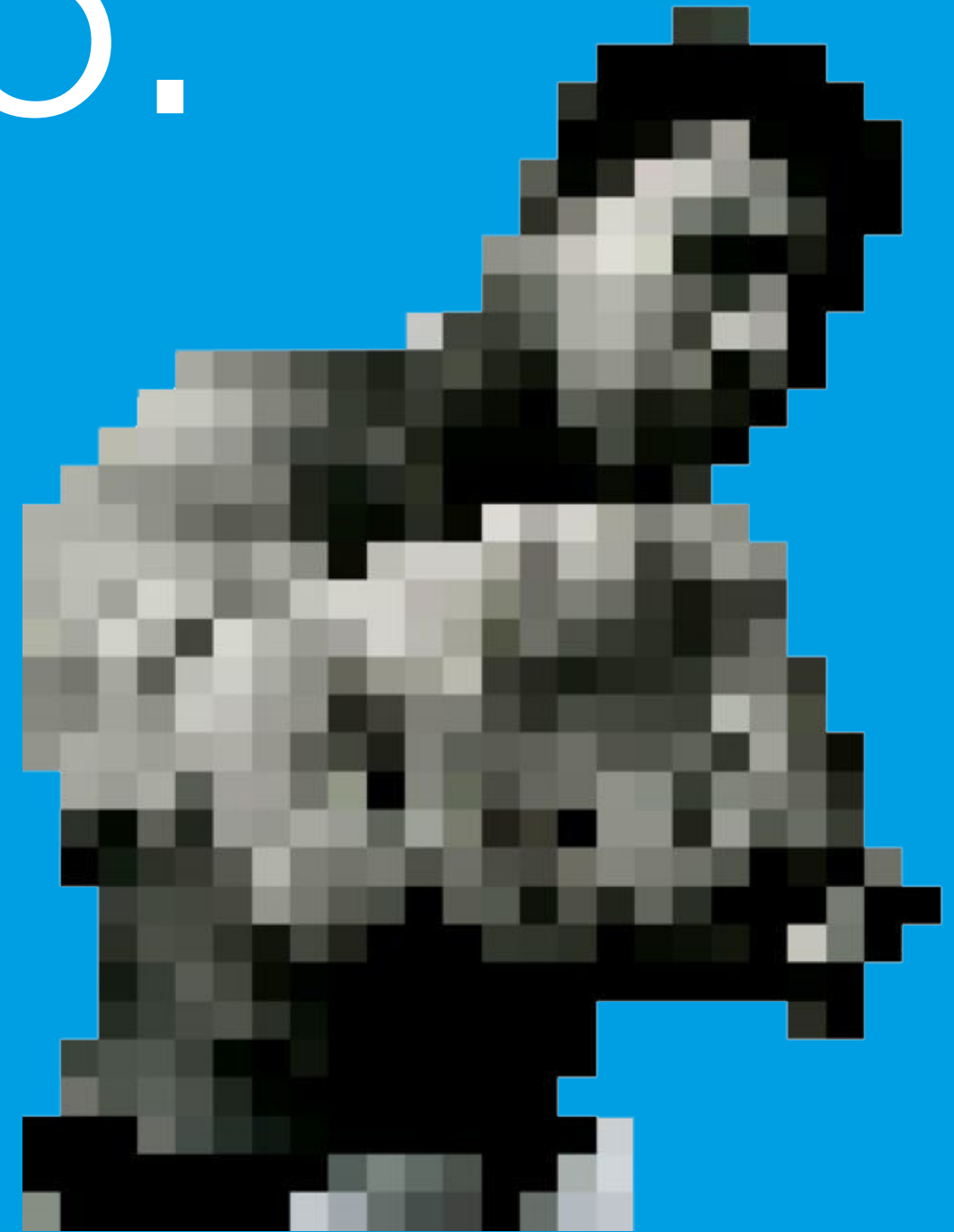
```
class CrunchEventListener : EventListener
{
    static readonly ActivitySource source = new("MyCompany.MyLibrary");

    protected override void OnEventSourceCreated(EventSource eventSource)
    {
        if (eventSource.Name == "MyCompany-MyLibrary")
        {
            EnableEvents(eventSource, EventLevel.Informational);
        }
    }

    protected override void OnEventWritten(EventWrittenEventArgs eventData)
    {
        using var activity = source.StartActivity("SomeWork");
    }
}
```



😬 Сценарий 5:
Компонент не
использует
ничего

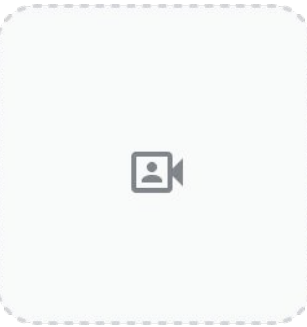


👉 Сценарий 5.1: Есть доступ к коду



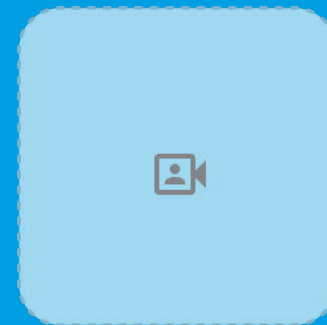
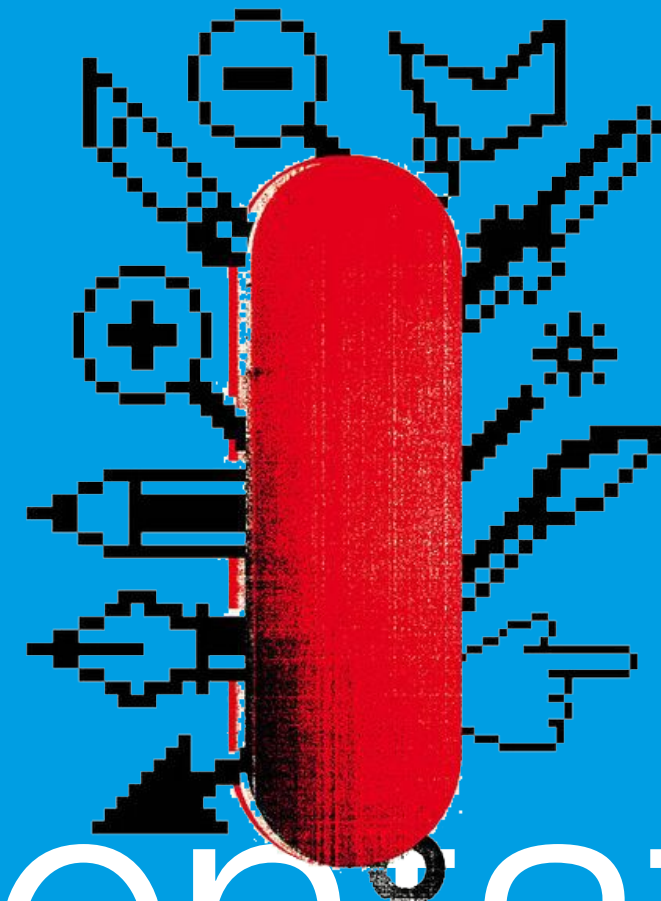
```
static readonly ActivitySource MySource = new("MyApp.Custom");  
  
using var activity = MySource.StartActivity("ProcessSomething",  
ActivityKind.Internal);  
activity?.SetTag("custom.key", "value");
```

👉 Сценарий 5.2: Нет доступа к коду



- `dotnet add [<PROJECT>] package OpenTelemetry.AutoInstrumentation`
- `Dockerfile`
- `OpenTelemetry Operator`





n

Auto-Instrumentatio

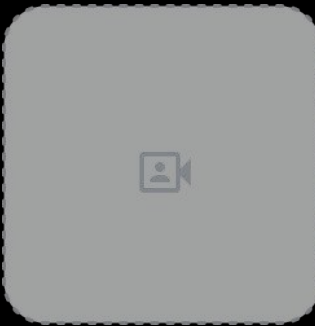
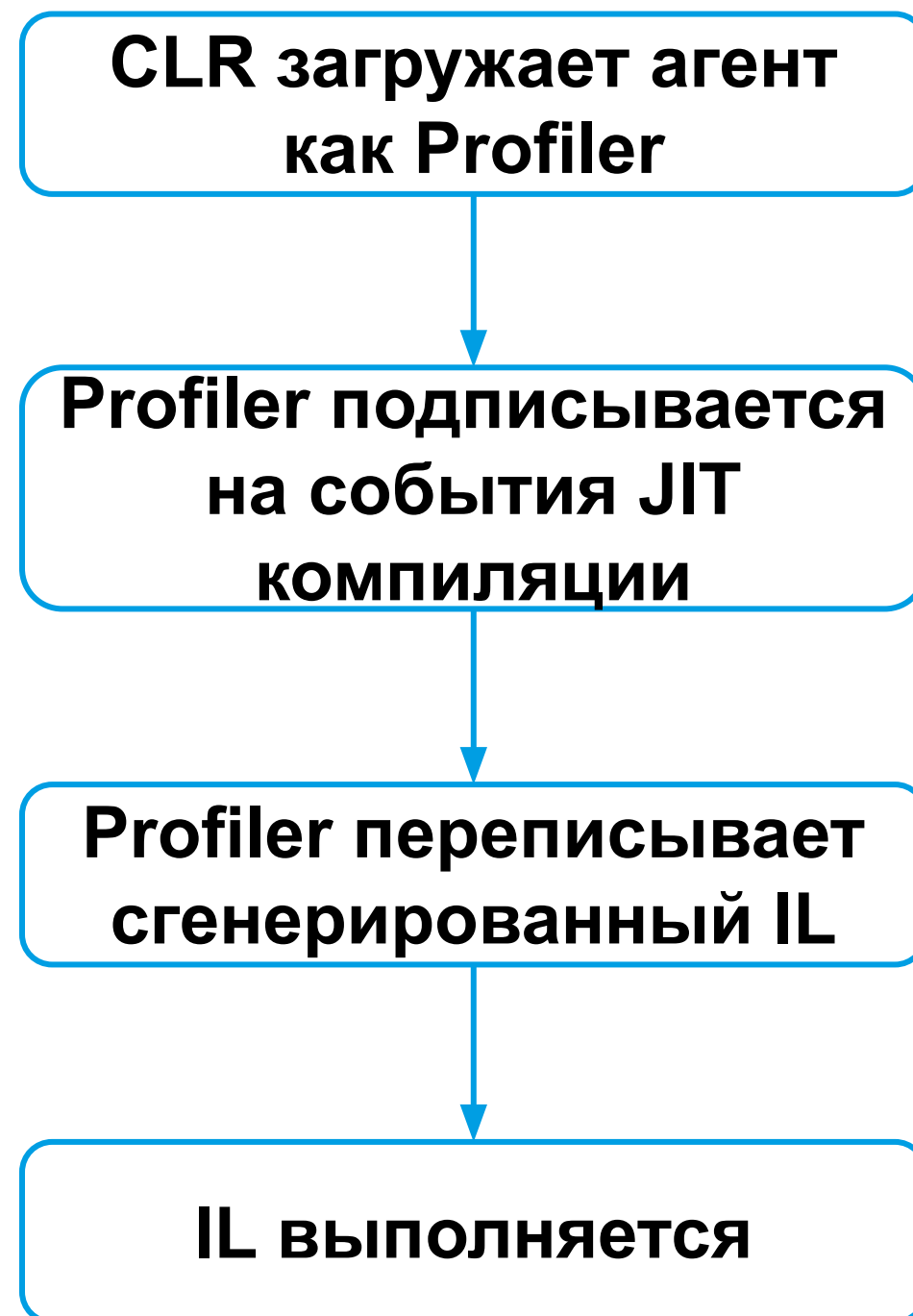




```
FROM mcr.microsoft.com/dotnet/sdk:8.0
# install OpenTelemetry .NET Automatic Instrumentation
ARG OTEL_VERSION=1.12.0
ENV OTEL_DOTNET_AUTO_HOME=/otel-dotnet-auto
ADD
https://github.com/open-telemetry/opentelemetry-dotnet-instrumentation/releases/download/v${OTEL_VERSION}/otel-dotnet-auto-install.sh otel-dotnet-auto-install.sh
RUN apt-get update && apt-get install -y unzip && \
    sh otel-dotnet-auto-install.sh
RUN chmod +x /otel-dotnet-auto/instrument.sh

WORKDIR /app
COPY . .
RUN dotnet publish -c Release -o out
```

IL injection

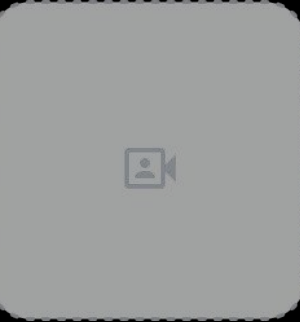


```
CORECLR_ENABLE_PROFILING=1
CORECLR_PROFILER={guid}
CORECLR_PROFILER_PATH=OpenTelemetry.AutoInstrumentation.Native.s
0
OTEL_DOTNET_AUTO_HOME=/opt/opentelemetry/instrumentation/net
```

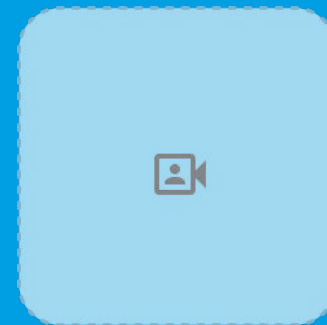


Что поддерживается?

- ASP.NET Core
- ASP.NET (System.Web)
- HttpClient / System.Net.Http
- gRPC
- SQL Client
- Entity Framework Core
- Redis (StackExchange.Redis)
- MongoDB.Driver
- etc...



Что обернуть в ActivitySource?



Отдельный ActivitySource

- Ваших библиотек транспорта
- Ваших shared-core библиотек
- Крупных подсистем

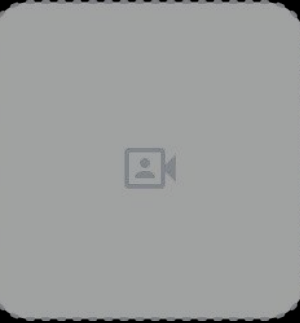
- `System.Data.SqlClient`
- `Microsoft.Data.SqlClient`
- `Azure.Messaging.ServiceBus`
- `Azure.Messaging.EventHubs`
- `System.Net.Http`



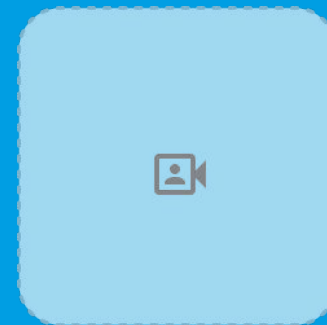
Отдельный Activity

- Получение
- Отправка
- Storage: БД запросы
- Internal
 - заметные стадии пайплайна

- . System.Net.Sockets.Socket.Connect
- . System.Net.Sockets.Socket.Accept
- . Grpc.Net.Client.GrpcOut
- . Grpc.AspNetCore.Server.GrpcIn
- . Azure.Messaging.ServiceBus.Send
- . Azure.Messaging.ServiceBus.Process
- . Azure.Messaging.EventHubs.Send
- . Azure.Messaging.EventHubs.Process
- . Microsoft.Data.SqlClient.Execute
- . System.Data.SqlClient.Execute



Sampling



Давайте трейсить вообще все!



- Google - \$0.20/million spans
- Azure - \$2.30 - \$2.76 per GB
- Amazon - \$0.50 per 1 million traces



Готовые стратегии Sampling



- `AlwaysOnSampler` — всегда записывать
- `AlwaysOffSampler` — никогда не записывать
- `TraceIdRatioBasedSampler(double ratio)`
- `ParentBasedSampler(Sampler root)`

```
builder.Services.AddOpenTelemetry()  
    .WithTracing(builder =>  
    {  
        builder  
            .AddSource("ComponentName")  
            .SetSampler(new AlwaysOnSampler())  
            .AddOtlpExporter()  
    });
```

Выводы



- Современные версии .NET прекрасно интегрируются с Otel
- Используйте готовые инструментации везде, где это возможно
- При необходимости можно легко дополнить свой код трейсами
- Если нет доступа к коду, то можно использовать Auto-Instrumentation

Что еще изучить?



- OpenTelemetry для самых маленьких
(<https://www.youtube.com/watch?v=X3faF3xw3m8>)
- OpenTelemetry Auto-Instrumentation
(<https://opentelemetry.io/docs/zero-code/dotnet>)
- .NET distributed tracing
(<https://learn.microsoft.com/en-us/dotnet/core/diagnostics/distributed-tracing>)

Спасибо!



@nikolay_savenko